

Save 和 Delete 的分层逻辑

先看大图：用户动作不会直接碰数据库，而是经过 UI、Redux thunk、API、Azure Function、Service、Repository。

1. UI Page

做什么：用户点击按钮



2. Redux Thunk

做什么：处理异步和错误



3. Frontend API

做什么：发 HTTP 请求



4. Azure Function

做什么：HTTP 入口



5. Service

做什么：业务规则



6. Repository

做什么：数据存储



7. Redux Update

做什么：回写页面

为什么需要：每层只负责自己的事情，后面替换后端存储或加认证时不会牵一发动全身。

用户点击 Save 之后发生了什么

核心：页面只发出保存意图；thunk 负责异步；后端分 Function、Service、Repository 三层处理。

1. UI Page / 页面

做什么：用户点击 Save，页面 dispatch(savePreCheck())。

为什么需要：页面只表达用户意图，不应该自己处理 HTTP、错误和数据存储。

2. Redux thunk / 异步流程

做什么：读取当前 preCheckDraft，找到今天是否已有记录，然后等待 API 保存结果。

为什么需要：把异步请求、loading、saving、error 从页面组件里拆出去。

3. DTO Mapping / 数据翻译

做什么：PreCheckDetailsLog 转成 PreCheckDto；保存后再从 PreCheckDto 转回 PreCheckLog。

为什么需要：前端 slider 字段和后端 API 字段不完全一样，所以需要翻译层隔离。

4. Frontend API / 请求层

做什么：调用 savePreCheck(dto)，实际发 POST /api/precheck。

为什么需要：集中管理 URL、headers、base URL；后面 token 也主要改这里。

5. Azure Function / HTTP 入口

做什么：SavePreCheck 读取 JSON body；body 为空返回 400；成功返回保存后的 DTO。

为什么需要：Function 负责 request / response，不负责具体业务存储。

6. Service + Mapping / 业务层

做什么：SaveAsync(dto) 把 DTO 转 Model，调用 Repository 保存，再转 DTO 返回。

为什么需要：后续 validation、用户权限、日期规则应该放在这一层。

7. Repository / 数据层

做什么：SaveAsync(log) 当前用内存字典按日期保存，同一天覆盖，没有就新增。

为什么需要：以后换 Azure Table / Cosmos / SQL，主要替换 Repository。

用户点击 Delete 之后发生了什么

核心：删除必须等后端确认；成功后 Redux 再移除本地记录，避免假删除。

1. UI Page / 页面

做什么：用户选择一条记录，页面 `dispatch(deletePreCheckLog(id))`。

为什么需要：页面只告诉系统要删除哪条，不直接改后端数据。

2. Redux thunk / 异步流程

做什么：调用 `deletePreCheck(id)`，等待后端确认删除。

为什么需要：避免后端失败时前端假装已经删除。

3. Frontend API / 请求层

做什么：调用 `DELETE /api/precheck/{id}`。

为什么需要：Slice 不需要知道 URL 怎么拼，也不需要直接碰 `fetch`。

4. Azure Function / HTTP 入口

做什么：`DeletePreCheck` 校验 id；找不到返回 404；成功返回被删除的 DTO。

为什么需要：HTTP 状态码和 request 校验属于入口层责任。

5. Service / 业务层

做什么：`DeleteAsync(id)` 调用 `Repository` 删除，返回被删记录或 `null`。

为什么需要：以后可以加权限、审计、不可删除规则。

6. Repository / 数据层

做什么：`DeleteByIdAsync(id)` 找到 `log.Id` 后从存储里删除。

为什么需要：只有 `Repository` 关心数据到底存在内存、Azure Table 还是 Cosmos。

7. Redux fulfilled / 回写页面

做什么：删除成功后从 `savedPreCheckLogs` 移除 id；如果删的是今天，就重置 `draft`。

为什么需要：让 UI 和后端确认后的结果保持一致。